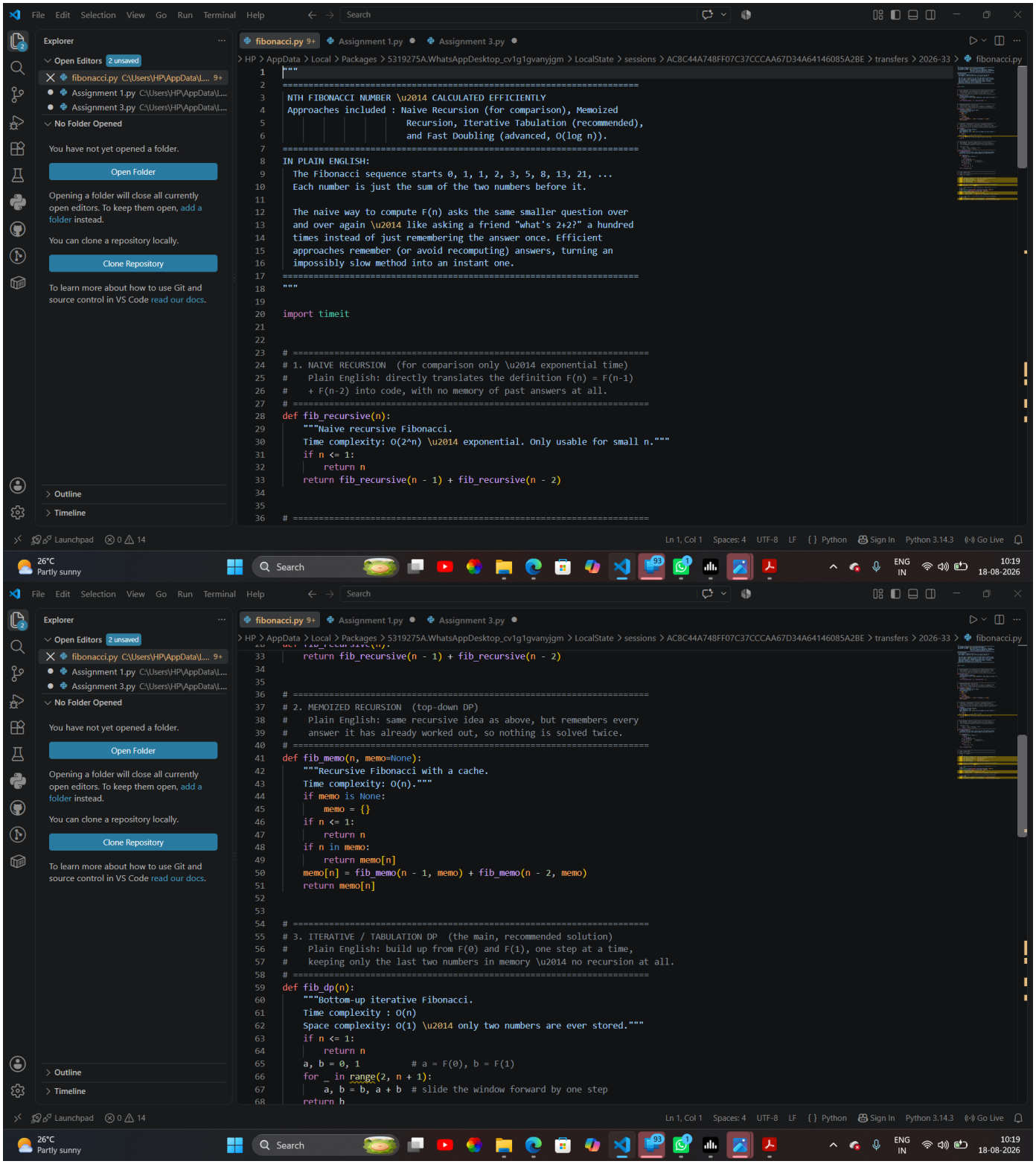
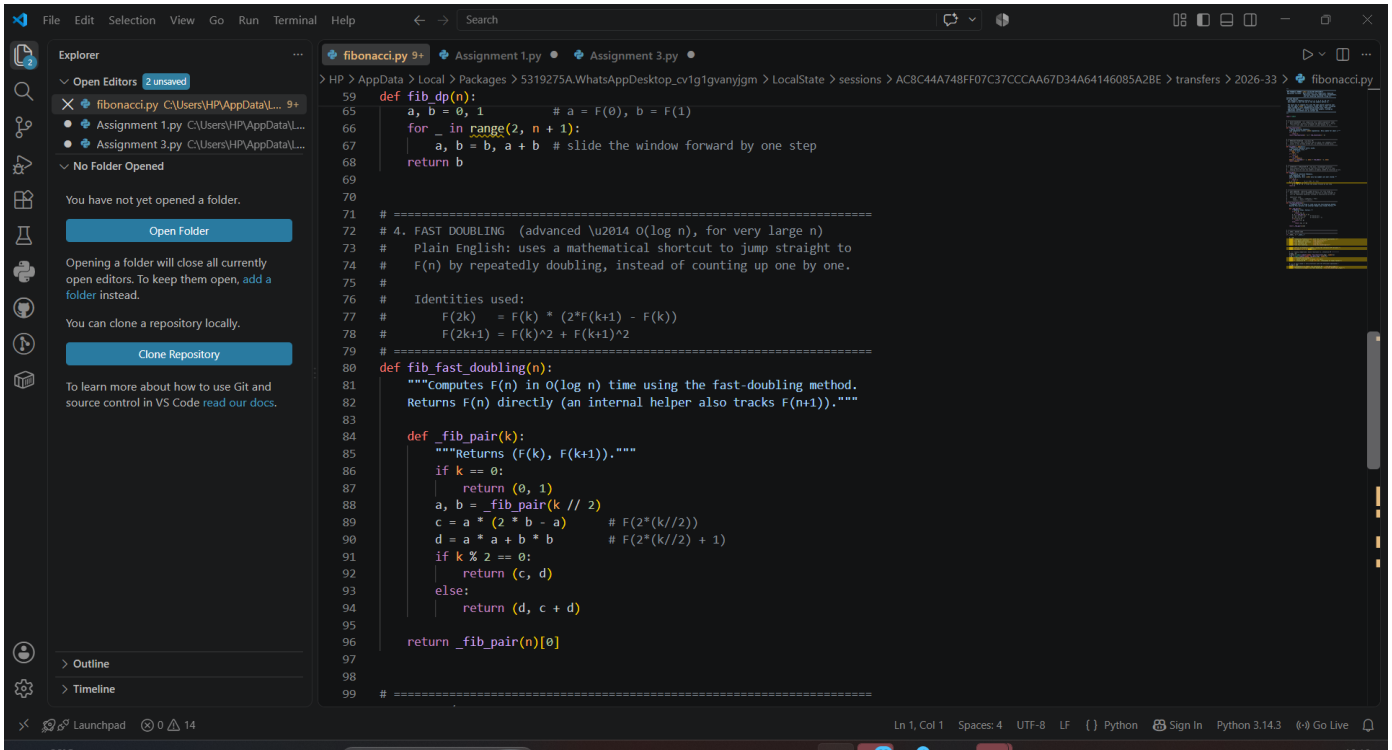
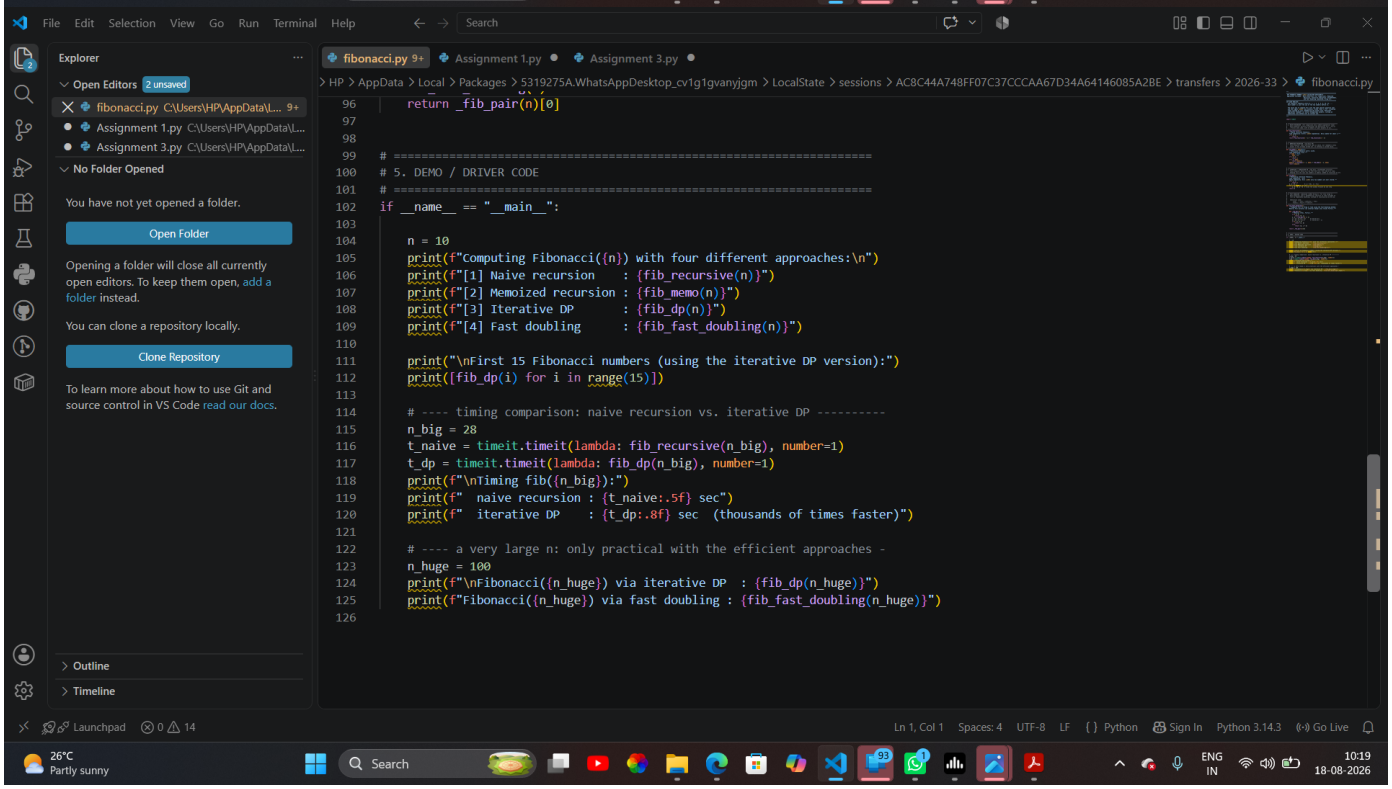
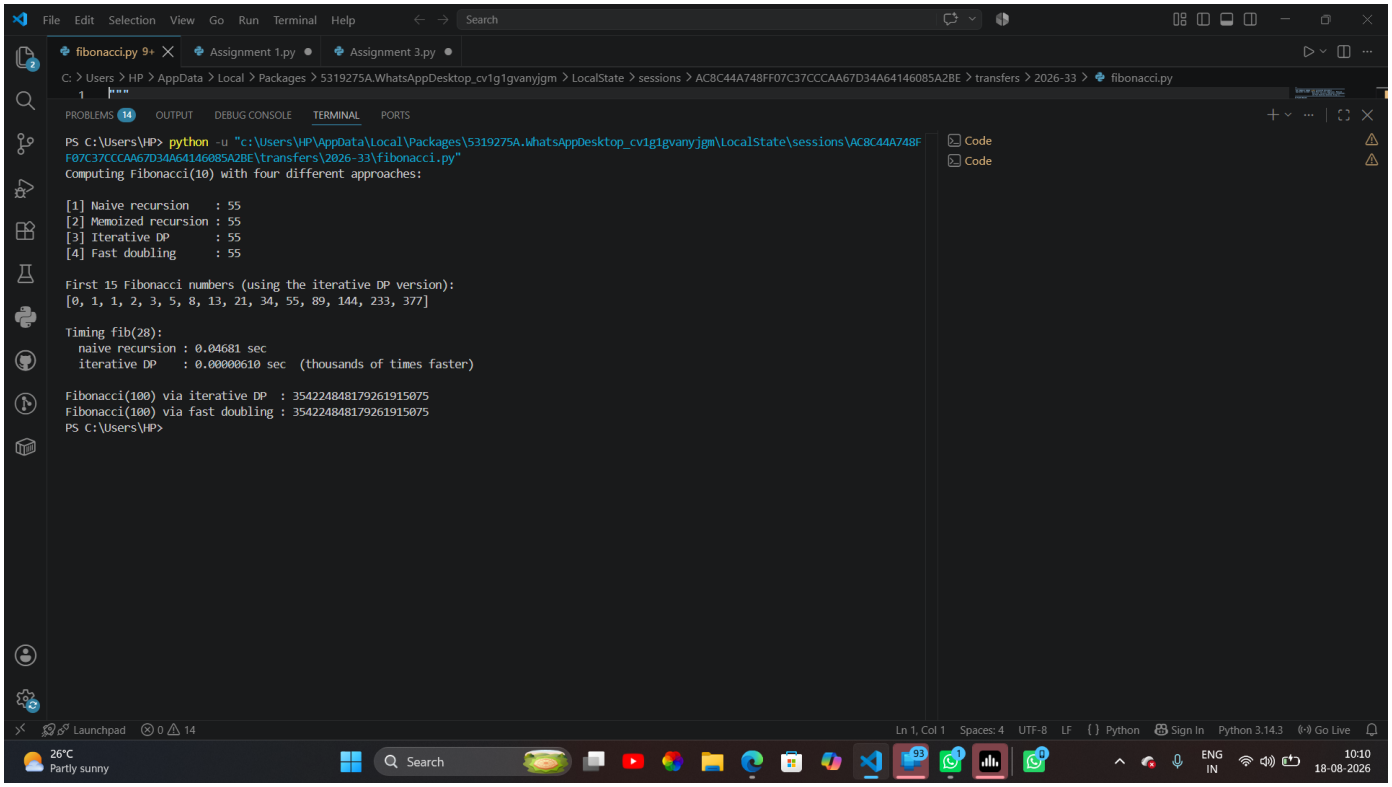

File Edit Selection View Go Run Terminal Help
Search
Explorer
Open Editors 2 unsaved
fibonacci.py C:\Users\HP\AppData\L... 9+
Assignment 1.py C:\Users\HP\AppData\L...
Assignment 3.py C:\Users\HP\AppData\L...
No Folder Opened
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
Outline
Timeline
fibonacci.py 9+
Assignment 1.py
Assignment 3.py
HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-33 > fibonacci.py
"""
NTH FIBONACCI NUMBER \u2014 CALCULATED EFFICIENTLY
Approaches included : Naive Recursion (for comparison), Memoized
Recursion, Iterative Tabulation (recommended),
and Fast Doubling (advanced, O(log n)).
IN PLAIN ENGLISH:
The Fibonacci sequence starts 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
Each number is just the sum of the two numbers before it.
The naive way to compute F(n) asks the same smaller question over
and over again \u2014 like asking a friend "What's 2+2?" a hundred
times instead of just remembering the answer once. Efficient
approaches remember (or avoid recomputing) answers, turning an
impossibly slow method into an instant one.
"""
import timeit
1. NAIVE RECURSION (for comparison only \u2014 exponential time)
Plain English: directly translates the definition F(n) = F(n-1)
+ F(n-2) into code, with no memory of past answers at all.
def fib_recursive(n):
"""Naive recursive Fibonacci.
Time complexity: O(2^n) \u2014 exponential. Only usable for small n."""
if n <= 1:
return n
return fib_recursive(n - 1) + fib_recursive(n - 2)
Launchpad 0 14
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Sign In Python 3.14.3 Go Live
26°C
Partly sunny
Search
ENG
IN
10:19
18-08-2026

File Edit Selection View Go Run Terminal Help
Search
Explorer
Open Editors 2 unsaved
fibonacci.py C:\Users\HP\AppData\L... 9+
Assignment 1.py C:\Users\HP\AppData\L...
Assignment 3.py C:\Users\HP\AppData\L...
No Folder Opened
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
Outline
Timeline
fibonacci.py 9+
Assignment 1.py
Assignment 3.py
HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-33 > fibonacci.py
return fib_recursive(n - 1) + fib_recursive(n - 2)
2. MEMOIZED RECURSION (top-down DP)
Plain English: same recursive idea as above, but remembers every
answer it has already worked out, so nothing is solved twice.
def fib_memo(n, memo=None):
"""Recursive Fibonacci with a cache.
Time complexity: O(n)."""
if memo is None:
memo = {}
if n <= 1:
return n
if n in memo:
return memo[n]
memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
return memo[n]
3. ITERATIVE / TABULATION DP (the main, recommended solution)
Plain English: build up from F(0) and F(1), one step at a time,
keeping only the last two numbers in memory \u2014 no recursion at all.
def fib_dp(n):
"""Bottom-up iterative Fibonacci.
Time complexity : O(n)
Space complexity: O(1) \u2014 only two numbers are ever stored."""
if n <= 1:
return n
a, b = 0, 1 # a = F(0), b = F(1)
for _ in range(2, n + 1):
a, b = b, a + b # slide the window forward by one step
return b
Launchpad 0 14
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Sign In Python 3.14.3 Go Live
26°C
Partly sunny
Search
ENG
IN
10:19
18-08-2026


File Edit Selection View Go Run Terminal Help
Search
Explorer
Open Editors 2 unsaved
fibonacci.py C:\Users\HP\AppData\L... 9+
Assignment 1.py C:\Users\HP\AppData\L...
Assignment 3.py C:\Users\HP\AppData\L...
No Folder Opened
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
Outline
Timeline
fibonacci.py 9+
Assignment 1.py
Assignment 3.py
HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-33 > fibonacci.py
def fib_dp(n):
a, b = 0, 1 # a = F(0), b = F(1)
for _ in range(2, n + 1):
a, b = b, a + b # slide the window forward by one step
return b
==
4. FAST DOUBLING (advanced \u2014 O(log n), for very large n)
Plain English: uses a mathematical shortcut to jump straight to
F(n) by repeatedly doubling, instead of counting up one by one.
#
Identities used:
F(2k) = F(k) * (2*F(k+1) - F(k))
F(2k+1) = F(k)^2 + F(k+1)^2
==
def fib_fast_doubling(n):
"""Computes F(n) in O(log n) time using the fast-doubling method.
Returns F(n) directly (an internal helper also tracks F(n+1))."""
def _fib_pair(k):
"""Returns (F(k), F(k+1))."""
if k == 0:
return (0, 1)
a, b = _fib_pair(k // 2)
c = a * (2 * b - a) # F(2*(k//2))
d = a * a + b * b # F(2*(k//2) + 1)
if k % 2 == 0:
return (c, d)
else:
return (d, c + d)
return _fib_pair(n)[0]
==
Outline
Timeline
Launchpad 0 14
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Sign In Python 3.14.3 Go Live
26°C Partly sunny
Search
ENG IN
10:19 18-08-2026


File Edit Selection View Go Run Terminal Help
Search
Explorer
Open Editors 2 unsaved
fibonacci.py C:\Users\HP\AppData\L... 9+
Assignment 1.py C:\Users\HP\AppData\L...
Assignment 3.py C:\Users\HP\AppData\L...
No Folder Opened
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
Outline
Timeline
fibonacci.py 9+
Assignment 1.py
Assignment 3.py
HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-33 > fibonacci.py
return _fib_pair(n)[0]
==
5. DEMO / DRIVER CODE
==
if __name__ == "__main__":
n = 10
print(f"Computing Fibonacci({n}) with four different approaches:\n")
print(f"[1] Naive recursion : {fib_recursive(n)}")
print(f"[2] Memoized recursion : {fib_memo(n)}")
print(f"[3] Iterative DP : {fib_dp(n)}")
print(f"[4] Fast doubling : {fib_fast_doubling(n)}")
print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
print([fib_dp(i) for i in range(15)])
---- timing comparison: naive recursion vs. iterative DP ----------
n_big = 28
t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
print(f"\nTiming fib({n_big}):")
print(f" naive recursion : {t_naive:.5f} sec")
print(f" iterative DP : {t_dp:.8f} sec (thousands of times faster)")
---- a very large n: only practical with the efficient approaches -
n_huge = 100
print(f"\nFibonacci({n_huge}) via iterative DP : {fib_dp(n_huge)}")
print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
Outline
Timeline
Launchpad 0 14
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Sign In Python 3.14.3 Go Live
26°C Partly sunny
Search
ENG IN
10:19 18-08-2026


File Edit Selection View Go Run Terminal Help
Search
fibonacci.py 9+
Assignment 1.py
Assignment 3.py
C: > Users > HP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-33 > fibonacci.py
1 """
PROBLEMS 14
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\HP> python -u "c:\Users\HP\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\AC8C44A748FF07C37CCCAA67D34A64146085A2BE\transfers\2026-33\fibonacci.py"
Computing Fibonacci(10) with four different approaches:
[1] Naive recursion : 55
[2] Memoized recursion : 55
[3] Iterative DP : 55
[4] Fast doubling : 55
First 15 Fibonacci numbers (using the iterative DP version):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
Timing fib(28):
naive recursion : 0.04681 sec
iterative DP : 0.00000610 sec (thousands of times faster)
Fibonacci(100) via iterative DP : 354224848179261915075
Fibonacci(100) via fast doubling : 354224848179261915075
PS C:\Users\HP>
Code
Code
Launchpad
0 14
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.3
Go Live
26°C
Partly sunny
Search
ENG
IN
10:10
18-08-2026